



NutriScan

Graduation Project Documentation

9-MONTH PROGRAM · INTAKE MAD-46 | ANDROID · iOS · BACKEND

Project	NutriScan
Version	1.1
Release period	13 July – 15 August 2026
Prepared by	Android, iOS, and Backend teams
Supervised by	Narden Ayad

TEAM

Android — Ahmed Tayseer · Ashraf Sherif · Nouraldeem Osama · Youssef El-Lban

iOS — Albaraa Alsayed · Ahmed Nageh · Osama Hosam · Mina Wagdy · Youssef Abd El-Fatah

Backend — Ibrahim Gad · Ibrahim Soliman · Reem Mohy

Acknowledgment

We would like to express our sincere gratitude to the Information Technology Institute (ITI) and the Ministry of Communications and Information Technology for providing the environment, the equipment, and the nine months of intensive training that made this project possible.

Our deepest thanks go to our supervisor, whose guidance shaped both the technical direction and the engineering discipline of this work, and to the instructors of the Native Mobile Application Development (Android and iOS) and Backend Web Development tracks, who taught us not only how to build for each platform but how to reason about architecture, testing, and maintainability across the full application stack.

We also thank our QA colleagues, Mina Mofreh and Ahmed Abouelella, who tested the application on real devices and produced the bug report that drove the final hardening round.

Finally, we thank our families for their patience during the long weeks of sprints, reviews, and late-night debugging.

Table of Contents

PART 1: Overview

Abstract

Introduction

Objectives

PART 2: Characteristics

Features

PART 2.1: SDLC

Analysis

Design

Implementation

Tools

PART 3: Illustrations

PART 3.1: Diagrams

ERD — Entity Relationship Diagram

Use Case Diagram

Class Diagram

PART 3.2: Screenshots

iOS Application

Android Application

Backend / Admin System

References

Abstract

Food labels are written for regulators, not for people. A shopper with a peanut allergy, a diabetic checking added sugar, or a parent buying a snack for a child with celiac disease is expected to decode an ingredient list printed in six-point type, in a language that may not be their own, while standing in a supermarket aisle. The information required to make a safe decision exists — it is simply not usable at the moment of the decision.

NutriScan is a cross-platform mobile application — available natively on both Android and iOS — that closes that gap. The user points the camera at a product: either its barcode or its ingredient panel. Within seconds the application returns a personalised verdict — Safe, Caution, or Unsafe — computed against that specific user's recorded allergies and chronic conditions, together with the exact ingredients that triggered the verdict and a full nutrition breakdown.

Around this core, NutriScan provides the daily-health context that makes a single scan meaningful: calorie and meal logging, water intake, step counting, a guided exercise library, a personalised nutrition news feed, a streak system, a configurable notification engine, and NutriGPT, a retrieval-augmented conversational assistant that answers nutrition questions in text or by voice and cites its sources.

The Android application is written entirely in Kotlin with Jetpack Compose, built on Clean Architecture across four Gradle modules with a strict MVI presentation contract, and is offline-first with a Room database as the single source of truth.

The iOS application is written entirely in Swift with SwiftUI, built on Clean Architecture with a feature-first module layout and an @Observable-driven MVVM presentation contract, and integrates with HealthKit for native step tracking.

Both clients consume the same NutriScan REST backend and authenticate against a self-hosted Keycloak identity provider using OpenID Connect. The backend is powered by a Java/Spring Boot service with an AI-driven scan analysis pipeline.

The final release is version 1.1, produced by a team of Android, iOS, and backend engineers over five sprints between 13 July and 15 August 2026.

Introduction

The problem

Chronic diet-related disease and food allergy are both rising, and both place the same demand on the individual: continuous, correct, per-product decisions.

A person with a severe allergy must verify every packaged product they eat. A single missed derivative name — casein for milk, semolina for gluten, E120 for an animal-derived colourant — is a clinical event, not an inconvenience.

A person managing diabetes, hypertension, or kidney disease must track sugar, sodium, and protein against a target they cannot compute mentally from a per-100g table.

Ingredient panels are dense, often bilingual, sometimes unreadable, and never personalised. They tell the reader what is in the product; they never tell the reader whether the product is safe for them.

Existing nutrition applications solve the adjacent problem — calorie counting — and treat safety as a manual search task. The user is still the one who must read, recognise, and decide.

The idea

NutriScan inverts that relationship. The user tells the application, once, who they are: their allergies, their chronic conditions, their body metrics, and the same information for their family members. From that point on, every product the user scans is evaluated against that profile, and the answer is delivered as a verdict a person can act on in one glance, before the product goes into the basket.

Two capture paths feed the same evaluation pipeline:

Barcode scan — on-device barcode detection reads the barcode from the live camera preview; the product is resolved from the NutriScan backend, falling back to the OpenFoodFacts open database.

Photo scan — the user photographs the ingredient panel; the image is uploaded and analysed server-side, returning the same structured result.

Both return a ScanResult: a verdict, a summary, a list of flagged ingredients with the reason each was flagged, and a normalised nutrition-facts block.

Beyond the scan

A verdict is a moment. Health is a habit. NutriScan therefore wraps the scanner in a daily-health loop: what the user ate (food log, fed directly from scan results), what they drank (water tracking), how much they moved (step counting plus a guided exercise library with per-exercise calorie estimates), and how consistently they did it (streaks and a notification engine that nudges without nagging).

The result is an application where a single scan is not an isolated lookup but an entry in a continuous personal health record.

Document structure

Part 1 states the project's abstract, motivation, and objectives. Part 2 details the delivered feature set and walks through the software development lifecycle for the Android client: requirements analysis, architectural and UI design, implementation of the key technical subsystems, and the tooling used. Part 2.2 covers the iOS client architecture and implementation. Part 3 summarises the backend. Part 4 presents the system diagrams and application screenshots, followed by references.

Objectives

Primary objective

Deliver production-quality native mobile applications — for both Android and iOS — that turn a packaged food product into a personalised safety verdict in under ten seconds, using only the device camera and a health profile the user configures once.

Product objectives

Personalised safety, not generic data. Every verdict is computed against the signed-in user's allergy and chronic-condition profile. The same product can be Safe for one account and Unsafe for another on the same device.

Two capture paths, one result. Barcode and ingredient-panel photo must produce the same structured ScanResult so that downstream features (history, saving, food logging) never need to know how a scan originated.

Explainability. The application never returns a bare verdict. It returns the specific flagged ingredients and why each was flagged, so the user can override an over-cautious result with informed judgement.

A complete daily-health loop. Calories, meals, water, steps, exercise, and streaks — so that a scan feeds a record rather than evaporating.

Family coverage. Allergy profiles are not only personal. A parent must be able to store family members with their own allergies and conditions.

Assistance on demand. NutriGPT answers free-form nutrition questions with cited sources, by keyboard or by voice, for users who cannot or prefer not to type.

Full bilingual parity. English and Arabic must be first-class: every string localised, full RTL layout mirroring, and an Arabic-appropriate typeface — not a translated afterthought.

Cross-platform parity. Android and iOS users receive the same feature set, the same verdict logic (server-side), and the same backend services.

Android engineering objectives

Clean Architecture with enforced boundaries. Four Gradle modules with a strict dependency rule (presentation → domain ← data) and a :domain module that is pure Kotlin with zero Android dependencies.

A single, uniform presentation contract. Every feature implements the same MVI triad (State / Event / Effect) with StateFlow for state and a Channel for one-shot effects.

Offline-first by default. The Room database is the single source of truth.

Testability as a delivery requirement. Every ViewModel ships with a test file covering state transitions, effects, and error paths.

Secure authentication. Standards-based OpenID Connect against a self-hosted Keycloak realm, with tokens held in encrypted storage.

Modern Android baseline. 100% Kotlin, 100% Jetpack Compose, no XML layouts, no Fragments, type-safe navigation, minSdk 31 / targetSdk 36.

iOS engineering objectives

Clean Architecture with feature-first modules. Each feature owns its Domain, Data, and Presentation layers; shared services live in Core.

@Observable-driven MVVM. Every screen is a stateless SwiftUI view driven by an @Observable ViewModel; all state flows through value-type state structs.

Coordinator-based navigation. AppFlowCoordinator owns top-level flow transitions (splash → onboarding → auth → profile setup → main tabs → account restoration); each tab maintains its own independent NavigationStack via AppRouter.

Native platform integration. HealthKit for real-time step counting; AVFoundation and the Speech framework for voice input and text-to-speech in NutriGPT voice mode; AVCaptureSession for barcode scanning.

Secure credential storage. Tokens stored in the iOS Keychain via KeychainManager; transparent token refresh handled by TokenRefresher at the network layer.

Modern iOS baseline. 100% Swift, 100% SwiftUI, @Observable macro, Swift Concurrency (async/await), minimum deployment target iOS 17.

Features

This section documents the implemented feature set of NutriScan v1.1, verified against the Android and iOS repositories.

1. Onboarding and authentication

Feature	Description
Splash	Branded launch screen; routes to onboarding, login, profile setup, or home based on session and profile state.
Onboarding carousel	Three-page introduction shown once; the “seen” flag persists in storage.
Register	Email/password registration against the NutriScan backend, with client-side validation and email-verification handoff.
Email verification	Verification-pending screen with resend support and rate-limit feedback.
Login	Email/password login.
Forgot password	Email-based password reset flow.
Profile setup pager	A multi-step flow collecting gender, date of birth, height, weight, chronic conditions, and allergies. BMI and TDEE are derived on completion.
Account deletion / restoration	Soft deletion with a pending-deletion state and a restore path within the grace window.

2. Scanning — the core feature

Feature	Description
Camera screen	A single camera-backed screen handling both modes. Barcode detection runs continuously in the live preview; the shutter captures the ingredient panel for photo analysis.
Barcode resolution	Barcode sent to NutriScan backend POST /v1/scans/barcode, falling back to the OpenFoodFacts public API.
Photo analysis	Image sent to POST /v1/scans (multipart) and returns a structured ScanResult.
Verdict	Every scan resolves to SAFE, CAUTION, or UNSAFE, computed server-side against the signed-in user's allergies and chronic conditions.
Product details	Verdict banner, product image and name, flagged-ingredient list with reasons, and a full nutrition-facts panel (calories, protein, carbohydrates, fat, fibre, sugar, sodium).
Add to food log	One tap adds the scanned product to today's meals with its calorie value.
Scan history	Full history with search, verdict filters (Safe / Caution / Unsafe), date filtering, and delete with confirmation.
Saved scans	Favourite/unfavourite any scan; saved scans sync to the backend.

3. Calories and daily tracking

Feature	Description
Calories dashboard	Daily calories consumed against the TDEE-derived target, with a macro breakdown.
Food log	Today's meals, added from scans or manually; removable with confirmation.
Water tracking	Cup-based intake against an adjustable daily target.
Step counting	Automatic step counting — Android uses a foreground hardware step-counter service; iOS uses HealthKit. Calories-burned derivation on both.
Calories history	Per-day historical breakdown (consumed, water, exercise, steps) with date navigation.
Step history	Historical step data with daily detail.
Streaks	Consecutive-day goal-completion streak, synced with the backend.

4. Exercises

Feature	Description
Exercise library	Backend-served exercise catalogue browsable by category, body part, equipment, and target muscle, with search and paging.
Workout screen	Animated GIF demonstration, step-by-step instructions, targeted and secondary muscles, and per-repetition / per-minute calorie estimates.
Workout logging	Completed workouts add their calorie burn to the day's tracking record.

5. News

Feature	Description
News home	Curated nutrition and health headlines on the home screen.
News list	Full article list with topic filtering, sourced from NewsAPI.
News detail	In-app article reading, with a web view for the full source.

6. NutriGPT assistant

Feature	Description
Chat	Free-form nutrition Q&A against a retrieval-augmented backend (POST /api/query). Answers list the sources they were drawn from.
Voice mode	Hands-free conversation: on-device speech recognition for input and text-to-speech playback of the answer, with an animated listening/speaking state.

7. Profile, family, and settings

Feature	Description
User profile	Personal details, body metrics, BMI, TDEE, allergies, and chronic conditions.
Edit profile	Full profile editing with avatar capture/upload.
Family members	Create, edit, and delete family members, each with their own allergy and chronic-condition sets and profile image.
App settings	Theme (light / dark / system) and language (English / Arabic), both persisted and applied instantly.
Notification settings	Per-type toggles plus quiet-hours windows.
Notification history	A persisted, readable log of every notification the application has raised, with read/unread state.
Help and Terms	In-app help content and terms & conditions.

8. Cross-cutting features

Bilingual EN / AR — full localisation in English and Arabic, with complete RTL layout support.

Light and dark themes — a semantic design-token system; no hard-coded colours in any view.

Accessibility-conscious UI — content descriptions/accessibility labels, shimmer/skeleton placeholders during loading, and standardised empty and error states across every screen.

PART 2.1: SDLC

This section presents the Software Development Life Cycle for the complete NutriScan system across Android, iOS, and Backend components.

Analysis

Development methodology

The project followed an Agile / Scrum process over five iterations — a Sprint 0 setup iteration followed by Sprints 1 to 3 and a final stabilisation and hardening phase — running from 13 July 2026 to 15 August 2026. Work was tracked on a Trello board (NutriScan, board 3j0IimeG) with six lists modelling the delivery pipeline:

List	Purpose
Product Backlog	All identified work, unscheduled.
Sprint Backlog	Committed scope for the active sprint.
In Progress	Actively being developed.
Bugs	Defects raised by QA or the team.
Code Review	Awaiting peer review on a pull request.
Done	Merged and verified.

Cards were prioritised with MoSCoW labels (must have, should have) and tagged by iteration (sprint 0, sprint 1, sprint 2, sprint 3) plus cross-cutting epics such as Auth integration. At delivery, 106 cards were in Done.

Sprint breakdown

Sprint	Focus	Representative delivered cards
Sprint 0 (20 cards)	Foundations and skeleton	GitHub repository setup, modularisation, Hilt DI setup, Retrofit network layer, Room database setup, Navigation Component setup, splash, onboarding, login/register/forgot-password UI, profile-setup pager, home screen, bottom navigation, localisation (EN/AR), search infrastructure.
Sprint 1 (9 cards)	Authentication and profile integration	Login, registration, and forgot-password backend integration via Keycloak; profile integration; scan integration; health-profile setup; edit-profile screen; settings; saved-scans screen.
Sprint 2 (9 cards)	Scanning and user data	Camera permission handling, product details, home/profile/settings data integration, profile refactor, exercises screen UI, application alert system.
Sprint 3 (6 cards)	Tracking and content	Calories screen, news screen, exercises backend integration, BMI/TDEE on profile, localisation fixes.
Stabilisation (62 cards)	Hardening and polish	Offline mode, barcode scanning, unit tests, push notifications and notification history, NutriChat AI, family-member CRUD and backend integration, RTL fixes, scan history, calories history, step details, empty states and shimmer placeholders, delete-user-data, UI unification, release v1.

Actors

Actor	Description
Registered User	The primary actor. Owns a health profile and performs every scanning, tracking, exercise, content, and assistant operation.
Family Member	A passive subject, not a login. A profile owned by a Registered User, carrying its own allergies and conditions.
NutriScan Backend	External system: scan analysis, product data, profile persistence, daily tracking, streaks.
Keycloak	External identity provider: OIDC authentication, token issue and refresh.
Third-party services	OpenFoodFacts (product fallback), NewsAPI (articles), Exercises API (catalogue), NutriGPT service (RAG answers).

Functional requirements

Account — register with email/password; verify email; log in with email/password or Google; recover a forgotten password; complete a health profile; view and edit the profile including avatar; manage family members; delete and restore the account; log out.

Scanning — scan a barcode; photograph an ingredient panel; receive a personalised verdict with flagged ingredients and nutrition facts; browse, search, filter, and delete scan history; save and unsave scans; add a scanned product to the food log.

Tracking — view daily calories against target; add and remove food-log entries; track water intake against an adjustable target; count steps automatically; view calories and step history by date; maintain a daily streak.

Exercise — browse and search the exercise catalogue by category; view exercise instructions and demonstration; log a completed workout and its calorie burn.

Content and assistance — read nutrition news; ask NutriGPT a question by text or voice and receive a cited answer.

Settings and notifications — switch theme; switch language; configure notification types and quiet hours; review notification history.

Non-functional requirements

Requirement	Target and approach
Offline availability	Every read path is served from Room. The application is fully usable with no connectivity; writes queue and replay.
Performance	Barcode recognition on-device via ML Kit; images loaded and cached through Coil; immutable collections in state to preserve Compose stability.
Security	OIDC/OAuth2 with PKCE via AppAuth; tokens in encrypted DataStore (security-crypto); automatic refresh at the OkHttp Authenticator layer; no credentials in source.
Localisation	100% string externalisation, English and Arabic, with layout mirroring verified per screen.

Requirement	Target and approach
Maintainability	Enforced layer boundaries, one uniform MVI contract, and module-scoped conventions documented in AGENTS.md.
Testability	98 test files; a ViewModel change without a corresponding test update is treated as incomplete.
Compatibility	minSdk 31 (Android 12) to targetSdk 36; phone form factors, portrait, light and dark.
Resilience	All repository calls wrapped in runCatchingCancellable, mapped to typed DomainExceptions and surfaced as standardised error states.

Backend and AI Analysis

The subsections below summarize how the backend and AI subsystems were analyzed, designed, implemented, and built, following the project's software development life cycle.

The core problem identified during analysis was that food-safety judgment is inherently personal, context-dependent, and error-prone when performed manually. Requirements analysis therefore centered on three pillars:

- Reliable extraction of product data from imperfect, real-world inputs such as photographed labels.
- Safety-decision process that can reason over incomplete data instead of failing outright.
- Backend capable of serving that decision instantly and repeatedly without incurring runaway AI inference cost.

This led to the following key functional requirements:

- Dual-path product ingestion (OCR and barcode),
- Personalized AI judge that consumes per-user health data
- Fallback web-search step for incomplete extractions
- Multi-member household profiles
- Daily nutrition tracking
- Full-text searchable scan history

Non-functional requirements included:

- Sub-second responses for repeated scans (met through caching)
- Resilience to partial infrastructure outages (met through the Elasticsearch → PostgreSQL fallback and RabbitMQ dead-letter queues)
- Secure multi-tenant access to sensitive medical data (met through Keycloak-issued JWTs and role-based access control)
- The ability to add or swap AI providers without rewriting business logic

Design

System Design

NutriScan uses a client-server architecture in which the native Android and iOS applications share the same REST backend and identity services. Both clients submit authenticated requests for profiles, scans, daily tracking, content, and assistant operations; the backend centralises business rules, persistence, AI orchestration, search, caching, and asynchronous processing.

This separation keeps safety verdicts and data contracts consistent across platforms while allowing each mobile client to use its native UI framework, navigation model, local persistence, and platform capabilities.

UI/UX Design

The Figma design system combines clinical clarity with a calm, approachable mobile experience. It uses restrained teal branding, generous spacing, soft rounded geometry, layered information hierarchy, and one visually dominant action at a time.

- Wireframes follow reusable patterns for onboarding, authentication and setup forms, the primary application shell, profile lists, detail/results, and empty or error states.
- Primary screens retain a persistent bottom navigation bar with a raised central scan action; deeper screens use contextual top navigation and a clear back action.
- Cards, pills, circular icon containers, and subtle teal-tinted elevation establish predictable interaction patterns without heavy borders.
- Light and dark themes preserve information order and component behavior; semantic color is reinforced with labels, icons, and placement rather than color alone.

Design element	Documented system
Primary color family	Teal scale from subtle #E8FAFA surfaces to deep #0F474A emphasis
Display typography	Plus Jakarta Sans for titles and subtitles
Interface typography	Lexend Deca for labels, body copy, metadata, and captions
Mobile spacing	Approximately 16–24 px outer margins with generous internal card padding
Accessibility	44–48 px touch targets, strong contrast, visible state labels, and logical focus order

●

Backend Design

The backend follows a layered architecture — Controller, Service, Repository — with DTOs and MapStruct mappers decoupling the public API contract from the persistence model. Cross-cutting, latency-sensitive work such as Elasticsearch indexing and cascading deletions is removed from the request path and handled asynchronously through RabbitMQ, publishing events only after the originating database transaction has committed (via @TransactionalEventListener) so that PostgreSQL and Elasticsearch never diverge.

The AI subsystem is designed around a Strategy pattern (a StructuredChatClient abstraction implemented separately for Gemini, Bedrock, and an OpenAI-compatible client) behind a Factory (AiConfig) that constructs the correct provider, and a Facade (AiService) that orchestrates the full OCR → judge → search-fallback pipeline behind a single-entry point. This keeps the pipeline provider-agnostic: a new LLM vendor can be added by implementing the same interface, without touching the orchestration logic.

iOS Design

Architecture Overview

The iOS NutriScan application is written entirely in Swift and SwiftUI, targeting iOS 17 and above. It follows a feature-first Clean Architecture: every feature owns its own Domain, Data, and Presentation sub-packages, while genuinely shared infrastructure lives in Core.

Top-level project structure

```

NutriScan/
├── NutriScan/
│   ├── NutriScan.xcodeproj
│   └── NutriScan/
│       ├── NutriScanApp.swift           @main entry point
│       ├── RootCoordinatorView.swift     Top-level flow switcher
│       └── Core/                         Shared infrastructure
│           ├── Navigation/               AppFlowCoordinator, AppRouter, Route
│           ├── Network/                  NetworkService, TokenRefresher, NetworkMonitor
│           ├── Security/                  KeychainManager
│           ├── Notifications/             SmartNotificationScheduler, NotificationService
│           ├── Shared Components/         Reusable SwiftUI views and controls
│           ├── SharedProfile/             UserProfileStore, shared profile use cases
│           ├── TabBar/                    MainTabView, AppTab
│           ├── DTOs/                     Common data-transfer objects
│           ├── Extensions/                Swift and SwiftUI helpers
│           ├── Helpers/                   AppValidator and utilities
│           ├── Constants/                 App-wide constants
│           └── Resources/                  Health quotes JSON
│       ├── DI/                           DIContainer, Assembly protocol, AppDependencies
│       ├── Features/                      One directory per feature (22 features)
│       └── Resources/
│           ├── Assets/                    Asset catalogue, custom fonts
│           └── Localization/               String tables (EN / AR)

```

Feature module structure

Every feature follows the same internal layout:

```

Features/<FeatureName>/
├── Domain/
│   ├── Entities/       Pure Swift value types – no framework imports
│   ├── Repositories/    Repository protocols
│   └── UseCases/         Use-case protocols and implementations
├── Data/
│   ├── DTOs/            Codable network response models
│   ├── Services/         Remote data sources / endpoint definitions
│   ├── Repositories/     Concrete repository implementations
│   └── Mappers/           DTO-to-Domain entity mappers
└── Presentation/
    ├── View/             SwiftUI views (stateless)
    ├── ViewModel/         @Observable ViewModels
    ├── Routing/           Feature-local routes
    ├── DI/                Feature Assembly registered in AppDependencies
    └── Extensions/        Presentation-only Swift extensions

```

The dependency rule matches Clean Architecture: Presentation → Domain ← Data. Domain types have no framework imports. Data types never appear in SwiftUI views.

The 22 feature modules implemented in the iOS app are:

Feature	Description
AccountRestoration	Pending-deletion screen and restore flow

Feature	Description
Auth	Login, register, forgot password, email verification
Calories	Daily calorie dashboard and food log
CaloriesHistory	Per-day historical calorie breakdown
EditProfile	Profile editing and avatar upload
Exercise	Exercise catalogue and workout player
Favorites	Saved/favourite scans
Home	Landing screen with health tips, news, and recent scans
News	Article feed with topic filters
NotificationHistory	Persisted notification log
NotificationSettings	Per-type notification toggles
Onboarding	Three-page first-launch introduction
PersonalInformation	Profile data display
ProductDetails	Scan result with verdict, nutrition, and flagged ingredients
Profile	Full profile view with family members
ProfileSetup	Multi-step onboarding health profile form
RAG	NutriGPT chat and voice assistant
Scan	Camera-based barcode and photo scanning
ScanHistory	Searchable and filterable scan history
Settings	Theme and language settings, help, terms
Splash	Animated launch screen
StepTracker	HealthKit step counting and history

Dependency injection

DIContainer is a thread-safe service locator. Each feature ships an Assembly struct that registers its use cases and repositories. All assemblies are composed in `AppDependencies.setup()`, called once at application launch from `NutriScanApp.init()`:

```
// Registration order (simplified)
CoreAssembly()           // NetworkServiceProtocol, KeychainManager
SharedProfileAssembly()  // UserProfileStore, FetchAndCacheProfileUseCase
StepTrackerAssembly()    // HealthKitStepDataSource, StepRepository use cases
NotificationAssembly()   // SmartNotificationScheduler, NotificationService
AuthAssembly()           // AuthRemoteService, Login/Register/ForgotPassword use cases
ScanAssembly()           // ScanRepository, Fetch/Submit/FetchDetail use cases
RAGAssembly()            // QueryRAGUseCase
// ... all other feature assemblies
```

Navigation — iOS

Top-level flow

RootCoordinatorView is the single root view. It holds an AppFlowCoordinator (ObservableObject) that tracks the active AppFlow:

Flow	Condition
.splash	App launch
.onboarding	First launch, hasSeenOnboarding == false in UserDefaults
.auth	Onboarding complete, no valid access token in Keychain
.profileSetup	Authenticated but hasCompletedProfileSetup == false
.main	Authenticated and profile complete
.pendingDeletion	Backend returns ACCOUNT_PENDING_DELETION (HTTP 409) on profile fetch

Transitions are animated via .animation(.default, value: flowCoordinator.flow). When the app enters .main the coordinator also triggers CaloriesActivitySyncCoordinator.synchronizePendingDates() and starts the NotificationBootstrapper.

Per-tab navigation

Inside .main, MainTabView renders five tabs (Home, Calories, Scan, Exercise, Profile). Each tab owns an independent NavigationStack managed by an AppRouter instance. Intra-tab navigation uses the type-safe Route / AnyRoute system: each feature declares its routes as an enum conforming to Route, and pushes them onto the tab's AppRouter stack.

Presentation layer — @Observable MVVM

All ViewModels use the @Observable macro (Swift 5.9 / iOS 17) rather than ObservableObject. Views are structs that read ViewModel properties directly; Combine publishers and @StateObject are not used.

A representative ViewModel shape:

```
@Observable
final class SomeFeatureViewModel {
    var uiState: SomeFeatureState = .idle
    var errorMessage: String?

    private let useCase: SomeUseCase

    init(useCase: SomeUseCase) {
        self.useCase = useCase
    }

    func handleAction(_ action: SomeAction) {
        Task {
            uiState = .loading
            do {
                let result = try await useCase.execute(action)
                await MainActor.run { uiState = .success(result) }
            } catch {
                await MainActor.run { errorMessage = error.localizedDescription }
            }
        }
    }
}
```



```
}
}
```

All async work runs in structured Task blocks; UI updates are always dispatched to MainActor.

Design Principles — iOS

Semantic colour tokens — all colours are resolved from named assets in the asset catalogue; no raw hex literals appear in SwiftUI views.

Font.AppFont — a shared typography system; no raw Font.system(size:) calls in feature views.

Shared components — reusable views (CachedImage, CachedAnimatedImage, CustomSearchBar, CustomPuffedButton, EmptyStates, FailureDialog, SuccessDialog, SearchSelectionSheet) live in Core/Shared Components and are used across features without duplication.

Dark-mode support — every colour set has both light and dark variants.

RTL support — layout direction adapts to the active locale; Arabic strings are served from the ar localisation table.

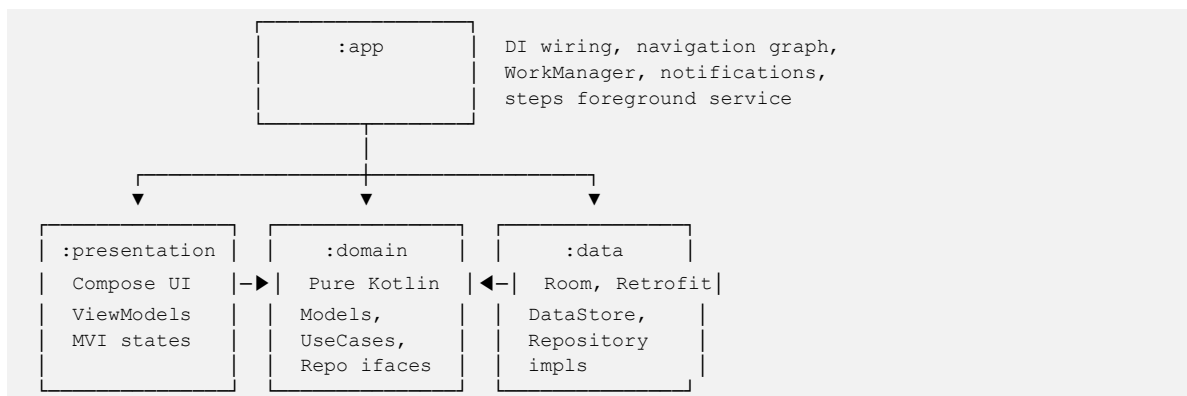
Accessibility — all interactive controls carry .accessibilityLabel, .accessibilityHint, and .accessibilityValue modifiers; loading states use shimmer-style placeholders.

Privacy — camera and HealthKit usage strings are declared in Info.plist.

Android Design

System architecture

NutriScan Android implements Clean Architecture across four Gradle modules:



The dependency rule: presentation → domain ← data. The :domain module is a plain java-library with no Android dependencies at all — no Context, no Uri, no Compose. :presentation and :data both depend on :domain and never on each other. :app is the only module that knows all three, and its sole job is to bind interfaces to implementations and host the navigation graph.

Presentation pattern: MVI

Every feature follows an identical contract:

```

feature/
├── state/
│   ├── FeatureState.kt    data class - everything the screen renders
│   ├── FeatureEvent.kt    sealed interface - everything the user can do
│   └── FeatureEffect.kt    sealed interface - one-shot side effects
└── viewModel/
  
```

```

├── FeatureViewModel.kt StateFlow<State> + Channel<Effect>
├── view/
│   ├── FeatureScreen.kt      stateless composable
│   └── components/          screen-local composables

```

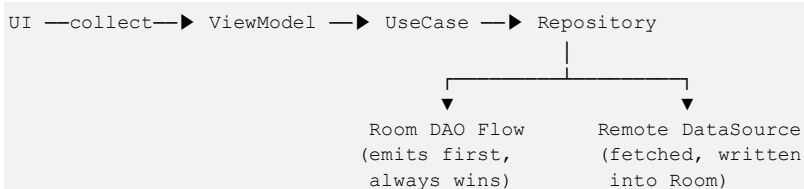
State is a single immutable data class exposed as `StateFlow<State>`. Collections inside it use `kotlinx.collections.immutable.ImmutableList` so that Compose can treat them as stable and skip recomposition correctly.

Event is the only way the UI talks to the ViewModel: a single `onEvent(event: FeatureEvent)` entry point.

Effect carries things that must happen exactly once — navigation, snackbars, opening the system settings — over a Channel.

Data layer design: offline-first

Room is the single source of truth. No screen ever reads the network directly.



Writes never block on the network. They are committed locally with tombstone flags — `pendingSync`, `deleted`, and `backendCreated` on `food_log` and `saved_scans`, `syncedToBackend` on `daily_tracking` — and replayed later by `DailyTrackingSyncEngine`, driven by a periodic `WorkManager` job every six hours.

Every repository call is wrapped in `runCatchingCancellable` and dispatched on an injected `@IoDispatcher`, converting throwables into typed domain errors while correctly propagating coroutine cancellation.

Database design

NutriScanDatabase (schema version 17) contains eleven tables:

Table	Role
users	The signed-in user's profile, metrics, allergy/disease ID sets, and family members.
diseases, allergies	Reference catalogues, seeded from the backend.
daily_tracking	One row per user per day: water, steps, calories burned, exercise minutes.
food_log	Individual logged meals with calories and verdict.
saved_scans	Scan results with verdict, flagged ingredients, and nutrition facts.
exercises, exercise_categories	Cached exercise catalogue.
workout_log	Per-day workout completion.
streak	Current consecutive-day streak per user.
notification_history	Every notification raised, with read state.

Family members are stored as a serialised JSON list on the users row rather than as their own table — a deliberate denormalisation, since a family member has no independent lifecycle, is never queried on its own, and always loads with its owner.

UI and visual design

The interface was designed in Figma before implementation and built with Material 3 on a custom design-token layer:

`AppTheme.colors` — a semantic palette (teal/cyan primary, verdict colours for safe/caution/unsafe) defined once for light and once for dark. Raw `Color(0xFF...)` values are prohibited in composables.

`AppTheme.typography` — a type scale with a Latin family and the Kosans family for Arabic; no raw `sp` values in feature code.

`AppTheme.shapes` — the corner-radius scale; no ad-hoc rounding.

Reusable components are shared rather than forked. Navigation is type-safe Compose Navigation: routes are `@Serializable` Kotlin objects and data classes. The shell is a five-tab bottom bar — Home, Calories, Scan (centre), Exercises/News, Profile — with animated tab and screen transitions.

RTL is a design constraint, not a post-process: layouts use start/end directional modifiers throughout, and `RtlUtils` centralises the cases where mirroring must be suppressed (numerals, charts, media controls).

Implementation

iOS Implementation

Networking

`NetworkService` is the single HTTP client. It builds `URLRequests` from `APIEndpoint` values, executes them with `URLSession`, decodes responses with `JSONDecoder`, and maps errors to `NetworkError`. `MultipartFormData` handles multipart uploads for scan images and avatars.

`TokenRefresher` sits above `NetworkService`. When a request fails with HTTP 401 it silently exchanges the stored refresh token at the Keycloak token endpoint, writes the new access and refresh tokens to `Keychain`, and retries the original request — feature code never sees the retry.

`NetworkMonitor` wraps `NWPathMonitor` to provide a reactive network-availability signal for offline-aware UI.

Security

`KeychainManager` provides save, get, and delete operations for typed `KeychainKey` entries (`.accessToken`, `.refreshToken`). All token I/O in the app goes through this class; no token ever touches `UserDefaults`.

Step tracking

The iOS step-tracking stack uses two data sources:

`HealthKitStepDataSource` — queries `HKStatisticsCollectionQuery` for historical step data by date range and observes `HKObserverQuery` for live updates during the current day.

`PedometerDataSource` — uses `CMPedometer` as a fallback for real-time step events when `HealthKit` is unavailable.

`StepRepositoryImpl` combines both sources behind the `StepRepositoryProtocol` domain interface. The repository is consumed by three use cases: `ObserveDailyStepsUseCase`, `RequestStepAuthorizationUseCase`, and `FetchStepsHistoryUseCase`.

Notifications

SmartNotificationScheduler schedules all local notifications via UNUserNotificationCenter.

SmartNotificationEvaluator consults the user's per-type preferences and quiet-hours windows (stored via NotificationMuteStore) before scheduling. NotificationService is the app-facing facade.

Types of notifications scheduled by the iOS app: - Water reminder - Step reminder - Streak reminder - Workout reminder - Food log reminder - Scan reminder - News digest - Health quotes - Break reminders

Every notification is also written to notification history storage so the user can review it in the Notification History screen.

NotificationBootstrapper.start() is called each time the app enters the .main flow; it re-evaluates and refreshes the notification schedule.

Authentication — iOS

Authentication follows the same OpenID Connect flow as Android, against the shared Keycloak realm nutriscan:

Email/password login — AuthRemoteService calls POST /v1/auth/login. On success, the response tokens are stored in Keychain via KeychainManager.

Email/password registration — multi-step form; client-side validation via AppValidator; backend call to POST /v1/auth/register.

Email verification — VerificationPendingViewModel supports resend via POST /v1/auth/resend-verification.

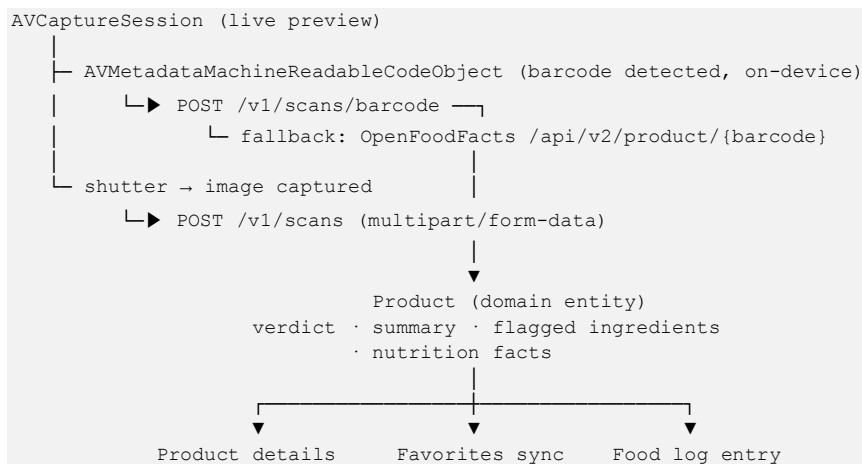
Forgot password — POST /v1/auth/forgot-password.

Token refresh — handled transparently by TokenRefresher at the network layer; no ViewModel is aware of the refresh.

Logout — AppFlowCoordinator.logout() deletes both tokens from Keychain, clears the in-memory UserProfileStore, resets tab navigation stacks, and transitions the flow to .auth.

Account restoration — if the backend returns ACCOUNT_PENDING_DELETION during profile fetch, the coordinator transitions to .pendingDeletion and presents AccountRestorationView.

The Scan pipeline — iOS



BarcodeScannerView is a UIViewRepresentable that wraps AVCaptureSession with AVCaptureMetadataOutput for on-device barcode decoding. When a code is detected the coordinator calls back the SwiftUI ViewModel without importing any UIKit type into feature code.

Both entry points (barcode and photo) converge on the same Product domain entity so that Scan History, Favorites, and the Food Log never branch on capture mode. Verdict evaluation is server-side — the allergen rule set is therefore updatable without shipping a new app release.

NutriGPT — iOS

Chat mode

RAGChatViewModel (@Observable) manages the conversation. It sends the user's text to QueryRAGUseCaseImpl, which calls POST /api/query, and appends the returned RAGMessage (containing answer and sources) to the displayed message list. The UI renders the answer text and a tappable source list.

Voice mode

RAGVoiceChatViewModel drives the hands-free experience:

State	Description
.idle	Microphone is inactive.
.listening	RAGSpeechRecognizerService (wrapping SFSpeechRecognizer) is transcribing live.
.thinking	Query sent, waiting for the backend response.
.speaking	AVSpeechSynthesizer is reading the answer aloud.
.error	Permission denied or recogniser unavailable.

Language is switchable at runtime (RAGLanguage.en / .ar); the recogniser is reconfigured without restarting the view. The ViewModel auto-detects the language of the spoken answer and synthesises it with the matching voice. Per the product requirement, neither the user's spoken question nor the assistant's answer is shown on screen — they are only spoken.

Profile setup — iOS

ProfileSetupFlowView presents a multi-step form collecting:

Gender

Date of birth

Height

Weight

Chronic conditions (selection from backend-served catalogue)

Allergies (selection from backend-served catalogue)

AppValidator enforces field rules (e.g. no future dates of birth, positive height and weight). On completion, BMI and TDEE are derived server-side and the flow transitions to .main.

Android Implementation

Authentication

Authentication is standards-based OpenID Connect against a self-hosted Keycloak realm (nutriscan):

Google sign-in runs through Keycloak's identity brokering using AppAuth with an authorisation-code flow and a nutriscan redirect scheme.

Email/password registration, verification, and reset run against the NutriScan REST backend (v1/auth/*), with Keycloak issuing the tokens.

Token storage is TokenManager, backed by DataStore with androidx.security:security-crypto encryption. Claims are parsed locally by JwtDecoder.

Token refresh is invisible to feature code. NutriScanAuthenticator, an OkHttp Authenticator, intercepts a 401 and transparently retries the original request after refreshing the token.

The scan pipeline



Both entry points converge on one domain model, so scan history, saved scans, and food logging never branch on capture mode. Verdict evaluation is server-side, computed against the profile carried on the authenticated request.

Daily tracking and synchronisation

daily_tracking is keyed by (userId, date). Water increments, step counts, and exercise minutes are written locally and marked unsynced. DailyTrackingSyncEngine reconciles local and remote state. Food-log entries carry pendingSync, deleted, and backendCreated flags so that a create-then-delete performed entirely offline resolves correctly without ever reaching the server as two operations.

Steps are read from the hardware step-counter sensor by StepsForegroundService (a FOREGROUND_SERVICE_HEALTH service). StepsSyncWorker uploads the totals.

Notifications

Nine @HiltWorker WorkManager workers cover water, steps, streak, workout, food, scan, news, quotes, and break reminders. Each consults the user's per-type preference and the configured quiet-hours window before posting, and every posted notification is recorded in notification_history.

NutriGPT

The assistant posts the user's question to a retrieval-augmented backend (POST api/query) and receives an answer plus its source list. Answers render through compose-markdown, and sources are shown as tappable references.

Voice mode is built on the Android platform APIs — SpeechRecognizer for input and TextToSpeech for output — wrapped behind a VoiceManager domain interface so the ViewModel never touches an Android type. Playback runs at 1.5x speed.

Localisation and RTL

Every user-facing string lives in strings.xml and values-ar/strings.xml (~607 entries each). Language is a persisted user preference, applied to the composition locally rather than requiring a process restart.

Testing and quality assurance

Automated testing — 98 test files across all four modules, on JUnit 5, with MockK for fakes and Turbine for Flow and Channel assertions. Coverage spans ViewModels (state transitions, effects, and error paths), repositories, the DailyTrackingSyncEngine, NutriScanAuthenticator, JwtDecoder, notification slot logic, and Room DAOs as instrumented tests.

Manual QA — a dedicated testing round was carried out on physical devices (Oppo Reno 6 5G / Android 11, Xiaomi Mi Note 10 / Android 12) and a Pixel 8 emulator (Android 16), producing a formal bug report of 23 defects:

Severity	Count
Critical	10
High	10
Medium	1
Low	2

By type: 9 functional, 8 UI, 5 mixed UI/functional. 15 of the 23 were fixed for release. Representative defects and their resolutions:

ID	Defect	Resolution
Bug-06	All forgot-password recovery options triggered the email flow.	Fixed — unsupported options removed.
Bug-07	Password reset accepted a weak password with no validation.	Fixed at the backend validation layer.
Bug-08 / Bug-10	Date of Birth accepted future dates on setup and on edit.	Fixed — forward-date validation on both paths.
Bug-16 / Bug-17	Names accepted digits; height and weight accepted zero and negative values.	Fixed — field validation with localised error messages.
Bug-18	Re-login after logout re-prompted for the full health profile.	Fixed — profile state restored from the server on login.

ID	Defect	Resolution
Bug-19	"Grant Permission" did not open system settings after a camera-permission denial.	Fixed.
Bug-23	The water target could be raised but never lowered.	Fixed — target is now bidirectional.

Version control was GitHub with feature branches and pull-request review; the final release comprises 603 commits from five contributors, with releases distributed to testers through Firebase App Distribution.

Backend Implementation

Layered Service Implementation

The application is implemented in Java 21 on Spring Boot 4.1.0. The controller layer exposes eight REST controllers covering authentication, users, scans, daily tracking, allergies/diseases reference data, and admin operations. The service layer contains twelve service classes, including dedicated AI-orchestration services, and delegates persistence to Spring Data JPA repositories with specification-based dynamic queries for search and filtering.

AI Pipeline

Ingredient extraction and safety judgment are implemented through Spring AI clients wired to Google Gemini (OCR, safety judge, and search-triggering), AWS Bedrock (an alternative structured-chat provider using Claude/Sonnet), and an OpenAI-compatible client. Tavily is integrated as a callable tool that the Gemini judge invokes autonomously when the extracted product data is insufficient for a confident verdict, and OpenFoodFacts is used directly as a REST lookup for barcode-based scans.

Event-Driven Processing

RabbitMQ implements a topic-exchange topology (scan.exchange) with dedicated queues for indexing, retry, scan deletion, and user deletion, each backed by a dead-letter exchange and per-queue dead-letter queue so failed messages are retried and, ultimately, preserved for inspection rather than lost. Publisher confirms and mandatory returns give the producer positive acknowledgement that each event was delivered.

Caching Strategy

Redis 7 backs Spring's declarative @Cacheable caching for the most expensive operations in the system:

- AI judge verdicts
- Barcode-derived verdicts
- Tavily-sourced nutrition search
- Frequently read reference
- Profile data
- Allergies reference data

- Diseases / medical conditions reference data

each with its own TTL (for example, one day for AI judge results, three days for barcode verdicts, and fifteen days by default for static reference data). Cache keys are generated centrally to remain stable across invocations, and selective rules avoid caching incomplete or still-processing results.

Search & Observability

Elasticsearch 9.4.2 provides fuzzy, typo-tolerant full-text search (fuzziness enabled for queries of three or more characters) and edge n-gram-based autocomplete suggestions.

Indexing is entirely event-driven through RabbitMQ, keeping the index eventually consistent with PostgreSQL, if Elasticsearch is unavailable or returns no results, the search service transparently falls back to a PostgreSQL specification query so the API remains available.

Kibana 9.1.4 is layered on top of Elasticsearch for operational visualization of index health and scan data.

Identity & Access Management

Keycloak 26.6 is deployed as a custom-built image serving the nutriscan realm, extended with a bespoke email-HTTP service-provider interface that sends verification and password-reset email through the Resend HTTP API, and a branded FreeMarker theme for password-reset, and email templates. The Spring Boot application validates OAuth2/JWT bearer tokens issued by Keycloak on every secured request and uses the Keycloak Admin Client during startup and via a scheduled reconciliation job to keep local and identity-provider user records in sync.

Deployment & CI/CD

The services deployed included:

- NutriScan-App service
- PostgreSQL for the application
- Keycloak
- PostgreSQL for keycloak
- Redis
- RabbitMQ
- RabbitMQ Web UI
- Elasticsearch
- Kibana

All are deployed on Railway as separate managed services with persistent volumes and custom DNS (nutriscan.dev and auth.nutriscan.dev).

GitHub Actions runs a two-stage pipeline on every push and pull request:

- Lint job that enforces formatting with the Spotless Maven plugin,
- Build job that compiles the application into an executable JAR.

Tools

iOS Tools

Language and platform

Tool	Version	Purpose
Swift	5.9+	Sole implementation language
SwiftUI	—	Entire UI layer
Xcode	15.4+	IDE and build system
Swift Package Manager	—	Dependency resolution
iOS Deployment Target	17.0	Minimum supported OS

Application frameworks and libraries

Library / Framework	Purpose
SwiftUI	Declarative UI; all screens are pure SwiftUI
Observation (@Observable)	Reactive ViewModel state without Combine
Swift Concurrency (async/await)	All asynchronous work
HealthKit	Native step counting and history
AVFoundation	Camera capture, barcode decoding, speech synthesis
Speech framework	On-device speech recognition for NutriGPT voice mode
CoreMotion (CMPedometer)	Fallback step counting
UserNotifications	Local notification scheduling
Network framework (NWPathMonitor)	Network connectivity monitoring
Security framework (Keychain)	Secure token storage
AppAuth	OpenID Connect / OAuth2 with PKCE
Kingfisher	Remote image loading and caching
SwiftUI-Shimmer	Skeleton loading placeholders

Testing — iOS

Tool	Purpose
XCTest	Unit and UI test framework

Tool	Purpose
XCUITest	UI automation tests (NutriScanUITests)

Unit tests cover business logic, domain model mapping, input formatting validators, and repository behaviour. UI tests exercise core navigation flows. Camera, HealthKit, motion, microphone, and speech features require a physical device and are validated manually.

Development and collaboration tools — iOS

Tool	Use
Xcode	Primary IDE
Git / GitHub	Version control, pull-request review
Figma	UI/UX design reference
iOS Simulator (iPhone 16 family)	Primary development target
Physical iPhone	Camera, HealthKit, voice, notifications testing

Android Tools

Languages and platform

Tool	Version	Purpose
Kotlin	2.2.0	Sole implementation language
Java	11	JVM target
Android Gradle Plugin	9.2.1	Build system
KSP	2.2.0-2.0.2	Annotation processing (Room, Hilt)
Android SDK	min 31 / target 36 / compile 37.1	Platform baseline

Application frameworks and libraries

Library	Version	Purpose
Jetpack Compose (BOM)	2026.02.01	Entire UI; Material 3 + extended icons
Navigation Compose	2.8.3	Type-safe navigation with serializable routes
Hilt	2.59.2	Dependency injection (+ hilt-work, hilt-navigation-compose)

Library	Version	Purpose
Room	2.8.4	Local database, source of truth
Retrofit	3.0.0	REST client
OkHttp	5.4.0	HTTP, interceptors, token refresh
kotlinx.serialization	1.11.0	JSON and typed navigation routes
DataStore Preferences	1.1.1	Preferences (theme, language, onboarding, notifications)
security-crypto	1.1.0-alpha06	Encrypted token storage
WorkManager	2.10.0	Background sync and notification scheduling
CameraX	1.4.2	Camera preview and capture
ML Kit Barcode Scanning	17.3.0	On-device barcode recognition
Coil 3	3.0.4	Image loading (Compose, OkHttp, SVG, GIF)
AppAuth	0.11.1	OpenID Connect / OAuth2 with PKCE
Firebase (BOM)	33.7.0	Analytics, App Distribution
androidx.browser	1.9.0	Chrome Custom Tabs
compose-markdown	0.5.4	Markdown rendering for NutriGPT answers
android-image-cropper	4.6.0	Avatar cropping
kotlinx-collections-immutable	0.3.8	Compose-stable collections in state
core-splashscreen	1.0.1	Splash screen API
Timber	5.0.1	Logging

Testing — Android

Tool	Version	Purpose
JUnit 5	5.10.2	Test framework across all modules
MockK	1.13.10	Kotlin-native mocking
Turbine	1.1.0	Flow and Channel testing
kotlinx-coroutines-test	1.8.1	Coroutine test dispatchers
Espresso / Compose UI Test	—	Instrumented tests

External services and APIs

Service	Use
NutriScan REST backend	Scans, profile, daily tracking, streaks, family members

Service	Use
Keycloak	Identity provider (OIDC, realm nutriscan)
OpenFoodFacts	Public product database fallback
NewsAPI	Nutrition and health articles
Exercises API	Exercise catalogue with demonstrations
NutriGPT service	Retrieval-augmented Q&A

Development and collaboration tools

Tool	Use
Android Studio	Primary IDE
Git / GitHub	Version control, pull-request review, releases
Trello	Scrum board, backlog, sprint tracking
Figma	UI/UX design and design-system handoff
Postman	API exploration and contract verification
Firebase App Distribution	Test-build distribution to QA
Google Sheets	Bug reporting and QA tracking

Backend Tools

Backend & AI Technologies

Technology	Version	Purpose
Java	21 (Temurin)	Core backend language
Spring Boot	4.1.0	Application framework
Spring Data JPA / Hibernate	—	Object-relational mapping
Flyway	Boot-managed	Versioned database migrations
Spring Security + OAuth2 Resource Server	—	JWT validation, role-based access control
Keycloak	26.6	OAuth2 / OIDC identity provider
RabbitMQ (AMQP)	4	Event-driven messaging, DLX / DLQ
Redis + Spring Cache	7	AI result and reference-data caching
Elasticsearch	9.4.2	Full-text scan search and autocomplete
Kibana	9.1.4	Dashboards over Elasticsearch indices

Technology	Version	Purpose
Spring AI	2.0.0	Gemini, Bedrock, and OpenAI-compatible AI clients
Tavily	API	Web-search tool for nutrition-fact fallback
OpenFoodFacts	REST API	Barcode-to-product/ingredient lookup
Cloudinary	1.39.0	Image upload and hosting
Spring WebSocket (STOMP)	—	Real-time scan-status notifications
SpringDoc OpenAPI	2.8.5	Swagger UI / OpenAPI specification
Resend (via custom Keycloak SPI)	—	Verification and password-reset email
MapStruct	1.5.5	Bean-to-DTO mapping
Lombok	—	Boilerplate reduction

Infrastructure

Component	Version	Purpose
PostgreSQL	17 (Alpine)	Application and Keycloak relational data
Elasticsearch	9.4.2	Product / scan indexing
Kibana	9.1.4	Search observability
RabbitMQ	4 (management)	Event queues and dead-letter queues
Redis	7 (Alpine)	Spring Cache backend
Keycloak	26.6	Custom-built identity server (theme + SPI)
Railway	—	Managed hosting and custom DNS

Build, CI/CD & Local Orchestration

Tool	Detail	Purpose
Maven (with wrapper)	—	Build tool
Spotless Maven Plugin	Eclipse style	Code formatting enforcement
GitHub Actions	—	CI: lint job then build job
Docker & Docker Compose	—	Local infrastructure and application orchestration

PART 3: Illustrations

PART 3.1: Diagrams

System and Backend Architecture Diagrams

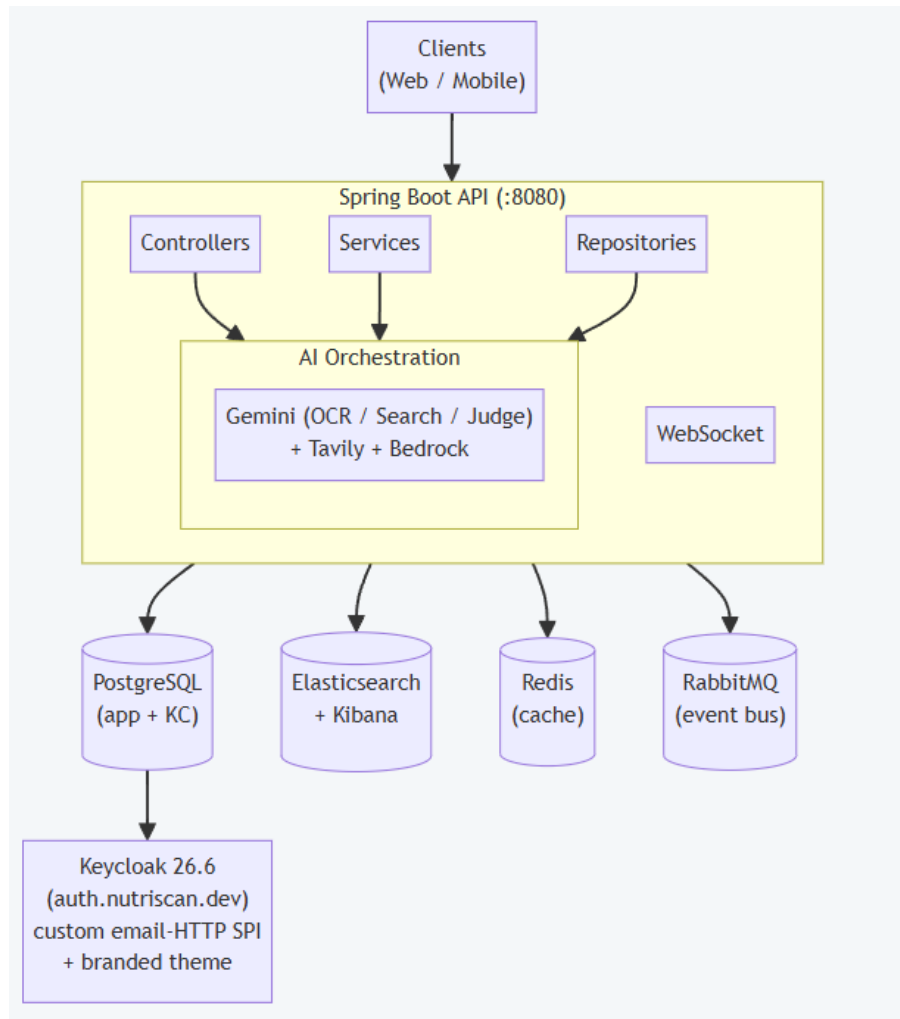


Figure 1. High-Level System Architecture

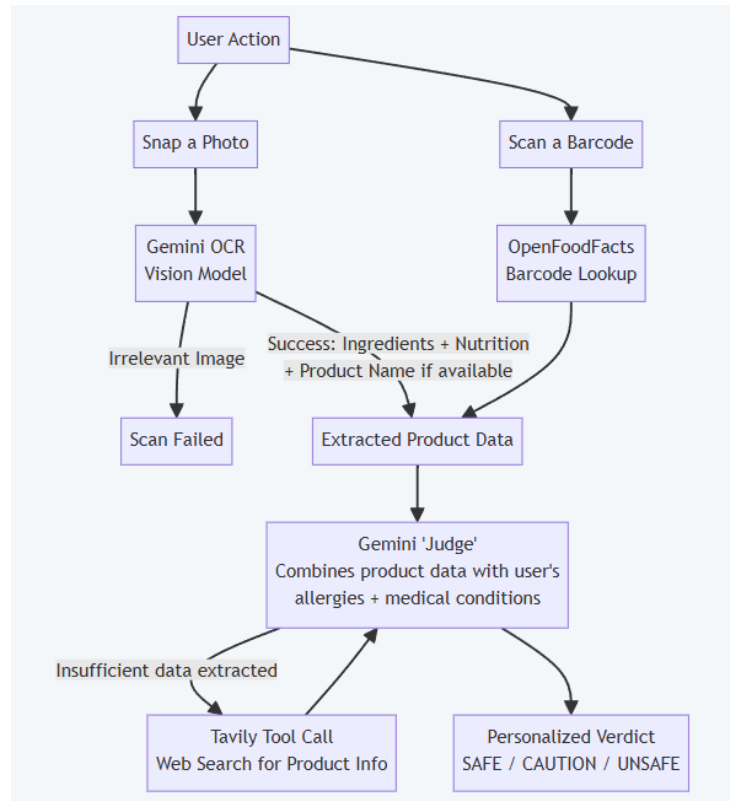


Figure 2. AI Food-Safety Pipeline

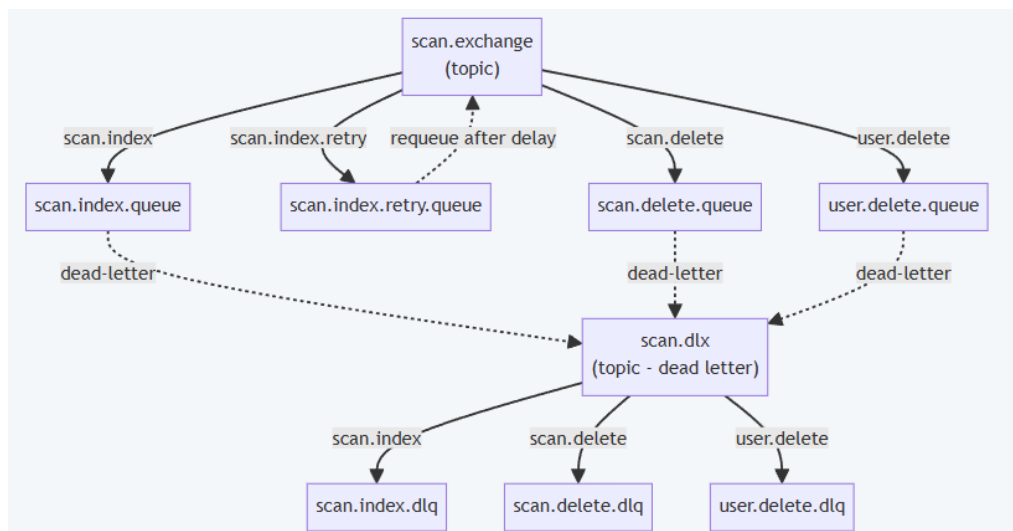


Figure 3. RabbitMQ Event Topology

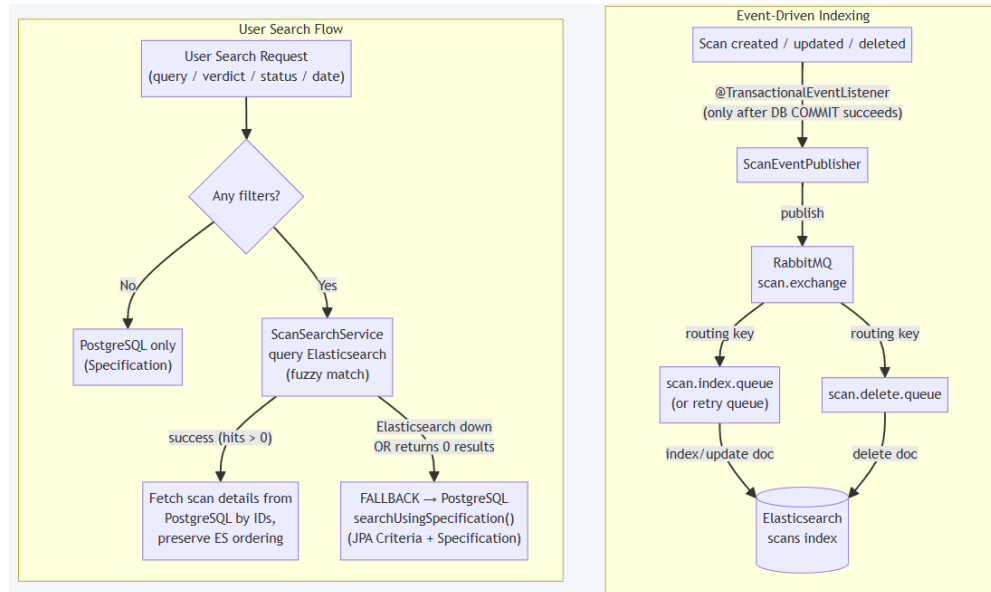


Figure 4. Search Flow: Elasticsearch, RabbitMQ, and PostgreSQL Fallback

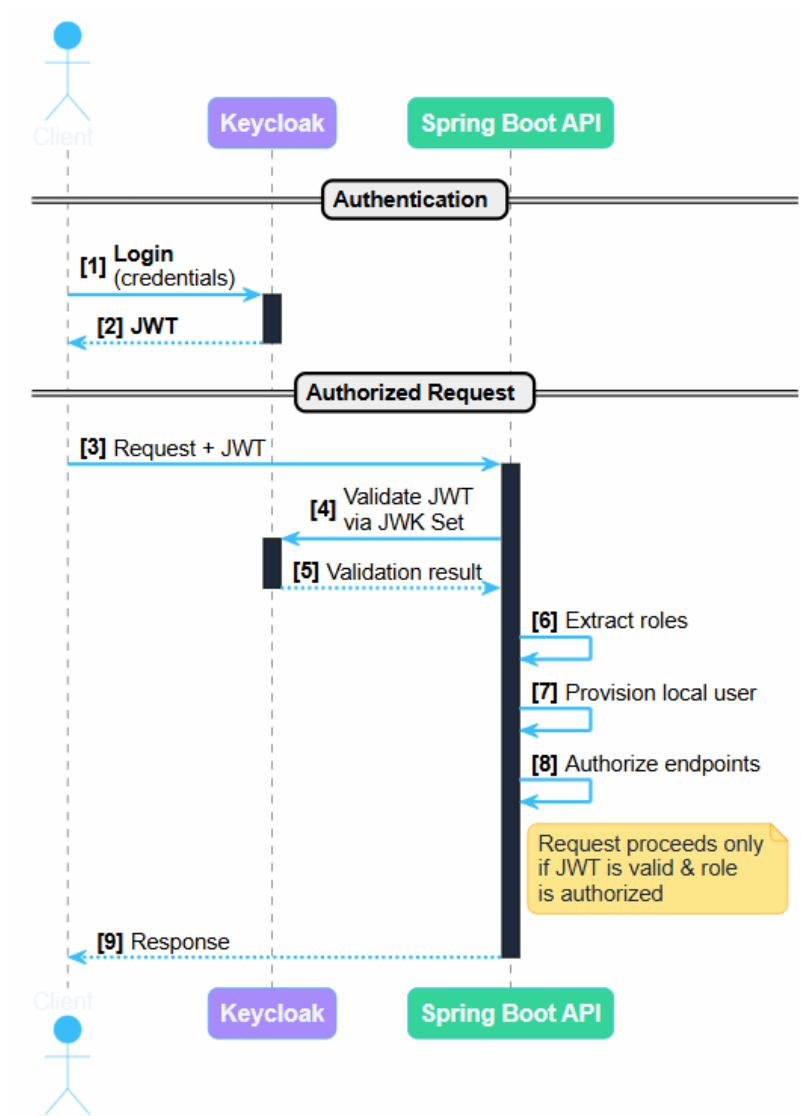


Figure 5. Authentication & Authorization Flow

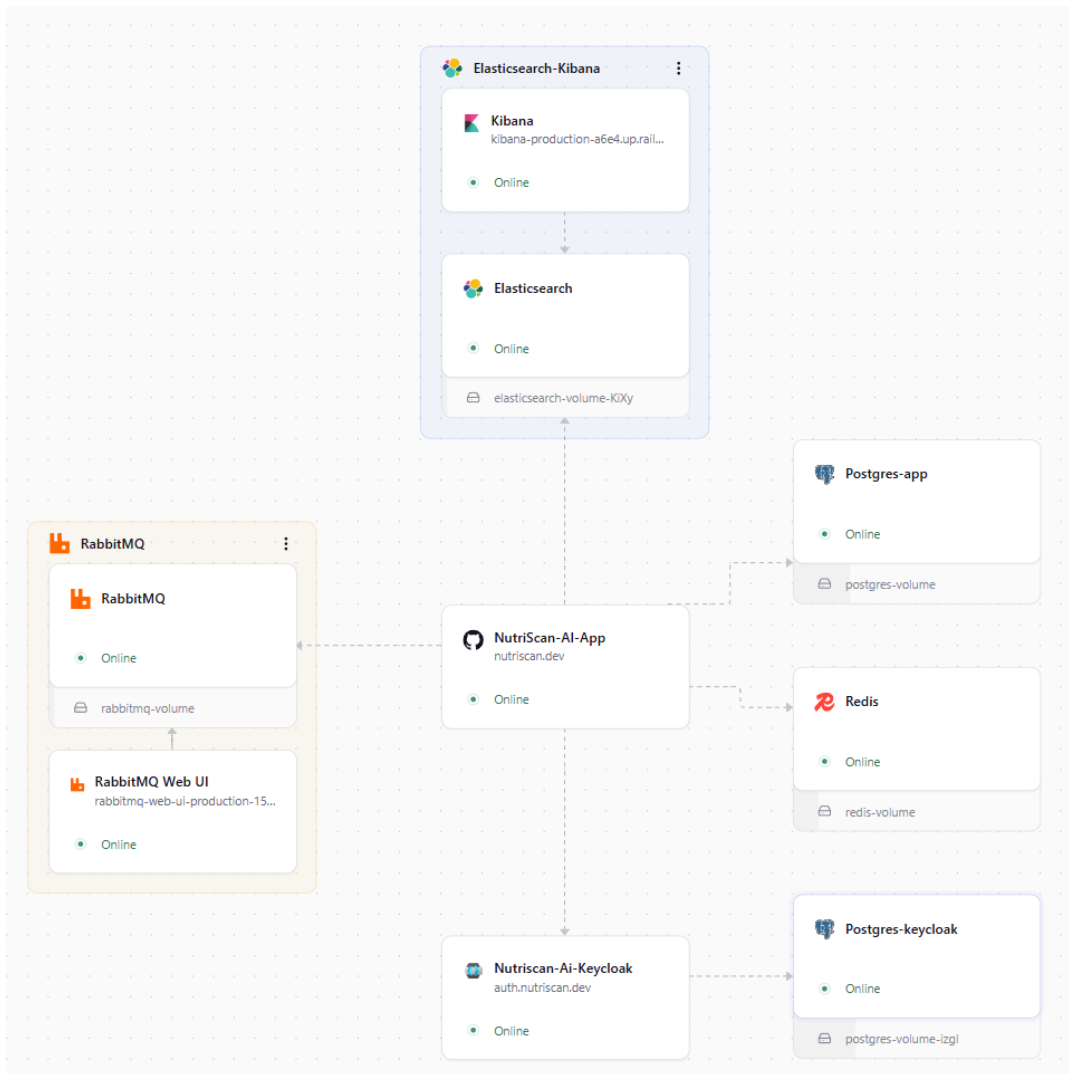


Figure 6. Railway Deployment Architecture

ERD — Entity Relationship Diagram

Database Schema

The application schema is managed and versioned by Flyway migrations. The domain model is organized around a central User entity with related entities for safety, nutrition, and family coverage:

- User & profile (users, profile data, user_allergy)
- Family members (family_member with independent family_member_allergy) verdict can be computed for each household member
- Scans (scan records with ingredients, verdict, flagged ingredients, and status)
- Daily tracking (daily_tracking and daily_tracking_meal) for per-day meal & nutrition aggregation.
- Search sync (elasticsearch_sync) for the ES to DB reconciliation and indexing bookkeeping.

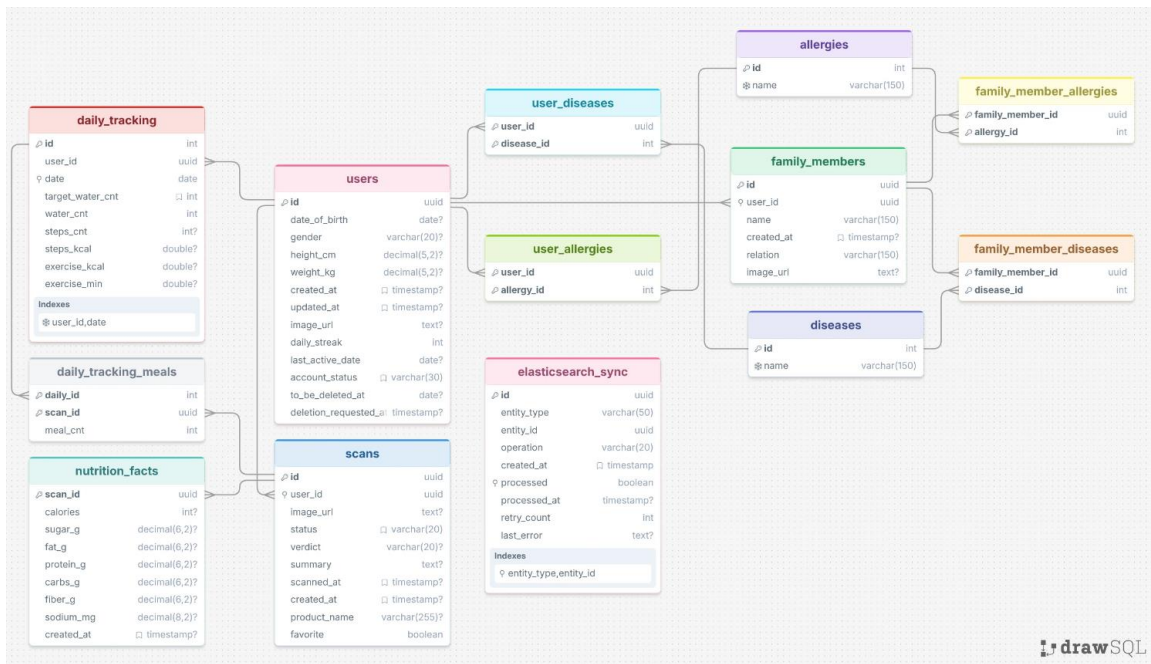


Figure 7. Backend database schema

The entity-relationship diagram of the local Android Room schema (version 17). **users** is the central entity; **daily_tracking**, **food_log**, **saved_scans**, and **streak** are all keyed by **userId**. **diseases** and **allergies** are reference catalogues referenced by ID sets on the user profile. **exercises** belongs to **exercise_categories**; **workout_log** records per-day completion.

The iOS client does not maintain a local relational database; it uses in-memory caching and **UserDefaults** for lightweight persistence, with the backend as the authoritative data store.

Use Case Diagram

The use-case diagram groups the application's capabilities into seven packages — **Account**, **Scanning**, **Tracking**, **Exercise**, **Content**, **Assistant**, and **Settings** — against the **Registered User** and **Family Member** actors, with the **NutriScan backend**, **Keycloak**, and the third-party APIs as external systems. The same diagram applies to both the Android and iOS clients, since the feature set is identical.

Class Diagram

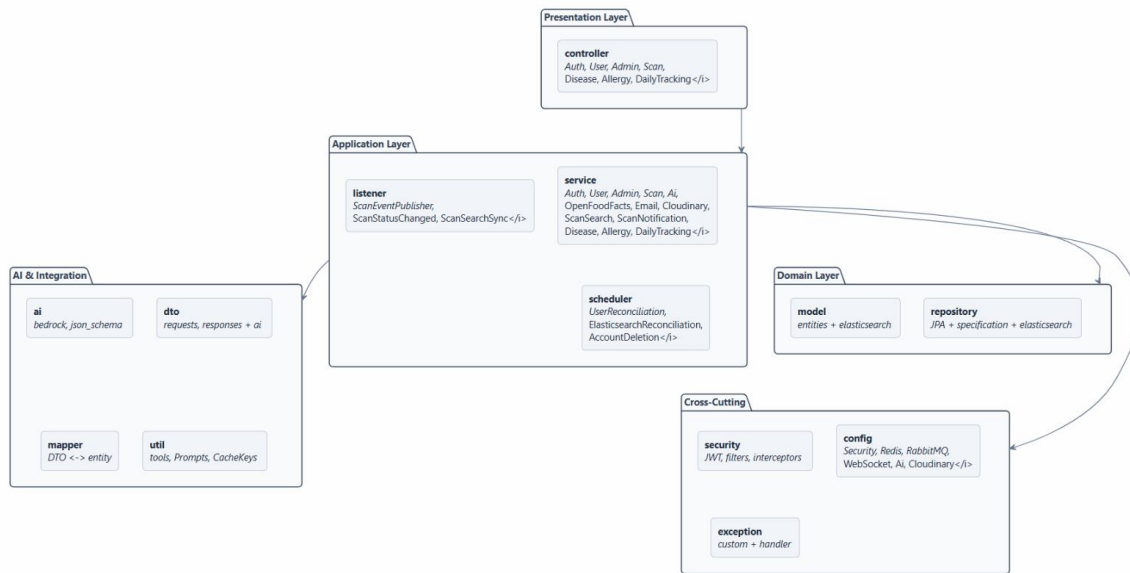


Figure 8. High level class diagram for backend structure

The Android class diagram shows the layered structure: domain models and repository interfaces at the centre, use cases depending only on those interfaces, data-layer implementations satisfying them, and a representative MVI triad (State / Event / Effect + ViewModel) on the presentation side.

The iOS class diagram mirrors this structure, replacing the MVI triad with an `@Observable` ViewModel and SwiftUI View, and replacing Hilt/Room with `DIContainer` / `URLSession`.

PART 3.2: Screenshots

iOS Application

The iOS application follows the shared product flows using native SwiftUI screens and iOS-specific navigation, HealthKit, camera, speech, and notification integrations.

Android Application

The Android application follows the same product flows using Jetpack Compose, CameraX, Room-backed state, type-safe navigation, and Android platform services.

Cross-platform Screenshot Register

#	Screen	Caption
1	Splash	Branded launch screen.
2	Onboarding	Three-page introduction to the application.
3	Login	Email/password sign-in with Google OIDC option.
4	Register	Account creation with validation.
5	Profile setup	Gender, date of birth, height, and weight collection.
6	Allergies / conditions	Health profile — the input to every verdict.
7	Home	Daily summary: calories, water, steps, streak, top news.
8	Scan — camera	Unified barcode and photo capture.
9	Product details — Safe	A product cleared against the user's profile.
10	Product details — Unsafe	Flagged ingredients with reasons.
11	Nutrition facts	Full macro and micro breakdown.
12	Scan history	Search, verdict filters, and date filtering.
13	Saved scans	Favourited products.
14	Calories	Daily target, macros, and the food log.
15	Calories history	Per-day breakdown over time.
16	Steps / step history	Automatic step counting and history.
17	Exercises	Catalogue by category with search.
18	Workout	Demonstration, instructions, calorie estimate.
19	News	Nutrition and health article feed.
20	NutriGPT chat	Cited answers rendered as Markdown.
21	NutriGPT voice	Hands-free assistant mode.
22	Profile	Metrics, BMI, TDEE, allergies, conditions.
23	Family members	Per-member allergy and condition profiles.
24	Notification settings	Per-type toggles and quiet hours.
25	App settings	Theme and language.

#	Screen	Caption
26	Arabic / RTL	The same screens fully mirrored.
27	Dark theme	Dark palette across key screens.

Backend / Admin System

No separate backend admin-interface screenshots were supplied. Backend behavior and infrastructure are represented by the architecture, deployment, ERD, and class diagrams in Part 3.1.

References

Project resources

Android source repository — <https://github.com/yusefellban/NutriScan>
 iOS source repository — <https://github.com/OTech-Company/NutriScan>
 Backend source repository — <https://github.com/ReemMohy259/NutriScan-AI>
 Android Project board (Trello) — <https://trello.com/b/3j0limeG/nutriscan>
 iOS Project board (Trello) — <https://trello.com/b/4tkFsGUD>
 UI/UX design (Figma) — <https://www.figma.com/design/zs5EkmcWPYqJvwWY5awVQc/NutriScan>

Platform and framework documentation

Android

Android Developers — <https://developer.android.com>
 Jetpack Compose — <https://developer.android.com/jetpack/compose>
 Guide to app architecture — <https://developer.android.com/topic/architecture>
 Room persistence library — <https://developer.android.com/training/data-storage/room>
 WorkManager — <https://developer.android.com/topic/libraries/architecture/workmanager>
 CameraX — <https://developer.android.com/training/camerax>
 Kotlin Coroutines — <https://kotlinlang.org/docs/coroutines-overview.html>
 Material Design 3 — <https://m3.material.io>

iOS

Apple Developer Documentation — <https://developer.apple.com/documentation>
 SwiftUI — <https://developer.apple.com/documentation/swiftui>
 Swift Concurrency — <https://docs.swift.org/swift-book/LanguageGuide/Concurrency.html>
 Observation framework — <https://developer.apple.com/documentation/observation>
 HealthKit — <https://developer.apple.com/documentation/healthkit>
 AVFoundation — <https://developer.apple.com/documentation/avfoundation>
 Speech framework — <https://developer.apple.com/documentation/speech>
 UserNotifications — <https://developer.apple.com/documentation/usernotifications>
 Swift Package Manager — <https://www.swift.org/package-manager>

Libraries and services

Android

Google ML Kit Barcode Scanning — <https://developers.google.com/ml-kit/vision/barcode-scanning>
 Dagger Hilt — <https://dagger.dev/hilt>

Retrofit — <https://square.github.io/retrofit>

OkHttp — <https://square.github.io/okhttp>

Coil — <https://coil-kt.github.io/coil>

AppAuth for Android — <https://github.com/openid/AppAuth-Android>

iOS

Kingfisher — <https://github.com/onevc/Kingfisher>

AppAuth for iOS — <https://github.com/openid/AppAuth-iOS>

SwiftUI-Shimmer — <https://github.com/markiv/SwiftUI-Shimmer>

Shared

Keycloak — <https://www.keycloak.org/documentation>

OpenFoodFacts API — <https://world.openfoodfacts.org/data>

NewsAPI — <https://newsapi.org/docs>

Firebase — <https://firebase.google.com/docs>

Concepts

Robert C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2017.

Model-View-Intent on Android — <https://developer.android.com/topic/architecture/ui-layer>

SwiftUI data flow and state — <https://developer.apple.com/documentation/swiftui/managing-model-data-in-your-app>

Offline-first application design — <https://developer.android.com/topic/architecture/data-layer/offline-first>

Mifflin-St Jeor equation (BMI / TDEE derivation) — American Journal of Clinical Nutrition, 51(2), 1990.

Backend

1. Spring Boot Reference Documentation — <https://docs.spring.io/spring-boot/>
2. Spring AI Reference Documentation — <https://docs.spring.io/spring-ai/reference/>
3. Keycloak Documentation — <https://www.keycloak.org/documentation>
4. RabbitMQ Documentation — <https://www.rabbitmq.com/docs>
5. Elasticsearch Reference — <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>
6. Redis Documentation — <https://redis.io/docs/latest/>
7. OpenFoodFacts API — <https://world.openfoodfacts.org/data>
8. Tavily Search API — <https://docs.tavily.com/>
12. Resend Email API — <https://resend.com/docs>
13. Railway Documentation — <https://docs.railway.app/>
14. NutriScan Project Repository (Backend) — <https://github.com/ReemMohy259/NutriScan-AI.git>